Институт искусственного интеллекта при МГУ имени М.В. Ломоносова
Лаборатория машинного обучения и семантического анализа

Корчагина Анна Вячеславовна

Модернизация, реконструкция и расширение пайплайнов проекта CMGN для построения карт знаний

Отчет по работе над проектом

Научный руководитель:
Ищенко Р. В.

Москва 2026

Содержание

1 Введение **3**

2 Цель, задачи, объект и предмет исследования **4**

2.1 Цель работы 4

2.2 Задачи работы 4

2.3 Объект и предмет исследования 4

2.4 Формальная постановка задачи 5

3 Методологическая основа и исследовательский подход **6**

4 Исходная архитектура проекта CMGN **6**

4.1 Часть модели работы с последовательностью 6

4.2 Графовая часть модели 7

4.3 Структура входа 7

4.4 Формальное описание обучения модели 8

5 Исследование форматов данных и реконструкция пайплайна **9**

5.1 Анализ train-данных 9

5.2 Анализ dev/test-данных 9

6 Построение кореферентных карт без AllenNLP **10**

6.1 Формальное описание кореферентной карты 10

7 Автоматическая разметка графов на основе DistilBERT **11**

7.1 Формальное описание DistilBERT-разметки 12

8 Пайплайн подготовки dev и test из сырых `.story`-документов **13**

8.1 Сборка полного dev/test pickle 13

8.2 Преобразование в seq2graph-вход 13

8.3 Оркестрация полного контура 14

8.4 Формальное описание преобразования документа в seq2graph-вход 14

9 Поддержка пользовательских dev-наборов и управляемого инференса **15**

10 Модификация `train.py` для работы с альтернативными dev-данными **16**

11 Система оценки качества и переработка метрик **16**

11.1 Sentence-level и keyword-level метрики 16

11.2 Диагностика проблем оценки 17

12 Интеграция TTED в общий оценочный контур **17**
12.1 Содержательный смысл TTED . . . 17
12.2 Как TTED реализован в текущем проекте . . . 18
12.3 Семантическая компонента TTED . . . 18
12.4 Формальное описание TTED в контексте проекта . . . 18
12.5 Что именно штрафует TTED . . . 19
12.6 Режим вычисления TTED . . . 20
12.7 Интерпретация результата . . . 20
12.8 Усредненный TTED по набору . . . 21

13 Внедрение RST-разметки как альтернативы DistilBERT **21**
13.1 Выбор парсера . . . 21
13.2 Реализация базового модуля RST-labels . . . 21
13.3 Логика directed-связей . . . 22
13.4 Формальное описание RST-labels на уровне предложений . . . 22

14 Принципы агрегации EDU-структуры на уровень предложений **23**
14.1 Формальная схема отображения EDU в предложения . . . 23

15 Анализ проблем, выявленных в ходе работы, и способы их устранения **24**
15.1 Проблемы зависимостей . . . 24
15.2 Проблемы данных . . . 24
15.3 Проблемы оценки . . . 25

16 Результаты проделанной работы **25**
16.1 Инженерные результаты . . . 25
16.2 Исследовательские результаты . . . 25
16.3 Экспериментальные измерения . . . 25
16.3.1 Метрики по статье . . . 26
16.3.2 Метрики по коду на test-наборе . . . 26
16.3.3 Метрики по коду на dev-наборе . . . 27
16.3.4 Замеры на разных вариантах dev-текстов . . . 27
16.3.5 Собственные карты кореферентности . . . 28
16.3.6 Собственный кодировщик для labels на обучении . . . 28
16.3.7 Сводная интерпретация замеров . . . 29
16.4 Практическая значимость . . . 30

17 Список ключевых модифицированных компонентов проекта **30**

18 Заключение **31**

1. Введение

Проект CMGN представляет собой исследовательскую программную систему, ориентированную на построение карты знаний по документу. В прикладной постановке задача формулируется следующим образом: по документу, состоящему из последовательности предложений, необходимо построить ориентированный граф управляющих связей между этими предложениями. Полученный граф затем используется в качестве структурной основы для построения карты знаний, отражающей ключевые смысловые отношения внутри документа.

Несмотря на наличие в проекте нейросетевой модели, служебных скриптов обучения и оценки, исходное состояние репозитория было недостаточно удобным для воспроизводимой работы с новыми данными. Существенная часть логики находилась не в одном связном пайплайне, а была распределена между историческими скриптами, промежуточными pickle-файлами, отдельными каталогами для train/dev/test и набором внешних зависимостей, часть из которых оказалась устаревшей. Это означало, что для полноценной работы с проектом требовалось не просто запускать модель, а реконструировать скрытую инфраструктуру: форматы данных, цепочку предобработки, принципы построения графов и механизм оценки качества.

В ходе работы проект рассматривался не как набор независимых файлов, а как исследовательская система, которую необходимо привести к воспроизводимому и управляемому состоянию. Основная деятельность была сосредоточена на следующих направлениях:

1. восстановление скрытого пайплайна подготовки данных;
2. замена устаревших компонентов, включая построение кореферентных карт;
3. разработка сценариев подготовки пользовательских dev/test-наборов из сырых `.story`-файлов;
4. внедрение и сравнение альтернативных источников supervision-разметки;
5. стабилизация и расширение системы оценки, включая TTED;
6. подготовка проекта к дальнейшим экспериментам в более открытой и модульной форме.

Настоящий отчет фиксирует не только конечные изменения в коде, но и логику их появления, проблемы, которые были обнаружены по ходу работы, и инженерные решения, принятые для их устранения.

2. Цель, задачи, объект и предмет исследования

2.1. Цель работы

Целью работы являлась реконструкция, модернизация и расширение проекта CMGN таким образом, чтобы обеспечить воспроизводимый и контролируемый полный цикл обработки текста: от сырых документов формата `.story` до получения структурных карт, запуска инференса, вычисления метрик качества и проведения сравнительных экспериментов с разными типами supervision-разметки.

2.2. Задачи работы

Для достижения поставленной цели были сформулированы следующие задачи:

1. изучить архитектуру проекта и назначение его ключевых модулей;
2. восстановить формат обучающих и тестовых сериализованных данных;
3. реконструировать пайплайн перевода исходного текста в формат, используемый моделью;
4. реализовать новый механизм построения кореферентных карт без AllenNLP;
5. внедрить устойчивую генерацию labels на основе DistilBERT;
6. обеспечить поддержку нескольких источников labels: gold, DistilBERT и RST;
7. создать управляемый dev-пайплайн для новых пользовательских `.story` файлов;
8. модифицировать обучение и инференс для работы с альтернативными dev-наборами;
9. расширить контур оценки, включая sentence-level, keyword-level и TTED-метрики;
10. внедрить RST-парсинг как альтернативный источник бинарной directed-разметки на уровне предложений;

2.3. Объект и предмет исследования

Объектом исследования является программный проект CMGN как система графового моделирования текста.

Предметом исследования являются:

- пайплайны подготовки train/dev/test данных;
- алгоритмы построения кореферентных и дискурсивных связей;
- способы автоматической генерации supervision-разметки;

- механизмы инференса и оценки структурных карт;
- способы интеграции внешней структурной информации в seq2graph-модель.

2.4. Формальная постановка задачи

Пусть документ задан в виде последовательности предложений

$$X = \{s_1, s_2, \ldots, s_N\}.$$

Требуется построить ориентированный граф

$$G = (V, E),$$

где

$$V = \{v_1, v_2, \ldots, v_N\},$$

а каждой вершине v_i соответствует предложение s_i.

В матричной форме искомый граф представляется матрицей связей

$$Y \in [0, 1]^{N \times N},$$

где элемент

$$Y_{ij}$$

характеризует степень того, насколько предложение s_i управляет предложением s_j.

Тогда задача CMGN может быть записана как задача построения отображения

$$f_\theta : X \mapsto \hat{Y},$$

где θ — параметры модели, а

$$\hat{Y} \in [0, 1]^{N \times N}$$

— предсказанная матрица управляющих отношений.

Дополнительно в графовую часть модели может подаваться внешняя структурная карта

$$G^{aux} = (V, E^{aux}),$$

например кореферентный граф, который используется как априорная структурная информация для GCN-компоненты.

3. Методологическая основа и исследовательский подход

Работа строилась на сочетании инженерного анализа кода, реконструкции данных и методологии прикладной обработки естественного языка. На первом этапе использовался статический анализ кодовой базы: исследовались файлы, структура каталогов, аргументы скриптов и форматы промежуточных артефактов. На втором этапе выполнялась реконструкция логики подготовки данных по косвенным признакам: по форме pickle-файлов, по использованию полей в `train.py` и `model_rl_gcn.py`, по поведению исторических скриптов и по взаимосвязи между train/dev/test каталогами.

Следующий этап носил экспериментально-инженерный характер. После восстановления общего контура системы были последовательно заменены и расширены отдельные компоненты: кореференция, supervision-разметка, dev-инференс, оценка и поддержка новых текстов.

Особое место заняла сравнительная логика supervision-сигналов:

- семантическая разметка на основе DistilBERT;
- gold-разметка из вручную подготовленных `.story` структур;
- риторическая directed-разметка на основе RST-парсинга.

Таким образом, работа сочетала в себе:

- reverse engineering существующего проекта;
- инженерную модернизацию кода;
- методическое расширение набора поддерживаемых источников разметки;
- отладку и верификацию корректности оценки.

4. Исходная архитектура проекта CMGN

Архитектурный центр проекта расположен в файле `model_rl_gcn.py`, где определен класс `Seq2Graph_rl_gcn`. Данная модель объединяет последовательностное кодирование текста и графовую обработку.

4.1. Часть модели работы с последовательностью

Вход документа представлен как набор предложений, каждое из которых состоит из последовательности слов, отображенных в словарные индексы. Для этих индексов загружается внешняя embedding-матрица, в которой слова уже отображены в векторные представления.

Затем применяются двунаправленные LSTM-слои:

- `lstm_for_sentence` кодирует предложение как последовательность токенов;
- `lstm_for_graph` используется в контексте дальнейшей графовой обработки.

Из этого видно, что модель не сводится к простому pairwise-классификатору между предложениями. Она сначала строит распределенные представления предложений, а затем использует графовый контекст для уточнения структуры связей.

4.2. Графовая часть модели

В проекте используется внешняя карта связей между предложениями, подаваемая в GCN. Она представляет из себя граф кореферентных связей, построенных на основе упоминаний сущностей. Функции и модули для работы с графом включают:

- модуль `GCNNet` обрабатывает graph-структуру;
- модуль `GCL_model` реализует дополнительный контур, связанный с contrastive/graph-consistency логикой;
- функции `gcn_learning`, `seq2graph` и `inference` обеспечивают основной путь вычисления модели.

4.3. Структура входа

Для корректной работы модель ожидает специфический формат данных, где каждый экземпляр содержит:

- идентификатор документа;
- padded-представление предложений;
- маски токенов;
- абстракт в индексном виде;
- матрицу labels;
- фактическую длину документа в предложениях;
- последовательностную маску;
- список исходных предложений документа;
- список предложений абстракта.

Именно жесткость этой структуры объясняет, почему в рамках работы пришлось не просто изменять одну функцию, а восстанавливать весь препроцессинг.

4.4. Формальное описание обучения модели

Пусть для документа с N предложениями модель предсказывает матрицу

$$\hat{Y} = (\hat{y}_{ij})_{i,j=1}^{N},$$

а supervision-разметка задается матрицей

$$Y = (y_{ij})_{i,j=1}^{N}.$$

После sentence-encoder и graph-модуля модель формирует скрытые представления предложений, на основе которых вычисляется матрица попарных взаимодействий, далее пропускаемая через сигмоиду:

$$\hat{y}_{ij} = \sigma(z_{ij}),$$

где z_{ij} — элемент матрицы межпредложенческих взаимодействий до нелинейности.

На диагонали предсказаний выполняется фиксация

$$\hat{y}_{ii} = 1,$$

что соответствует тривиальному совпадению предложения с самим собой.

Базовая supervised-потеря в текущей реализации задается среднеквадратичным отклонением:

$$\mathcal{L}_{sup} = \frac{1}{N^2} \sum_{i=1}^{N} \sum_{j=1}^{N} \left(\hat{y}_{ij} - y_{ij}\right)^2.$$

Кроме supervised-компоненты, в проекте присутствуют дополнительные члены функции потерь:

- RL-компонента $\mathcal{L}_{rl}$, связанная с выбором предложений и reward на основе ROUGE-сходства с абстрактом;
- GCL-компонента $\mathcal{L}_{gcl}$, реализующая дополнительную графовую регуляризацию.

Итоговая функция потерь в упрощенной форме записывается как

$$\mathcal{L} = \mathcal{L}_{sup} + \lambda_{rl}\mathcal{L}_{rl} + \lambda_{gcl}\mathcal{L}_{gcl},$$

где

$$\lambda_{rl} = \texttt{rl_lambda}, \qquad \lambda_{gcl} = \texttt{gcl_lambda}$$

— настраиваемые коэффициенты вклада соответствующих слагаемых.

В тестовой фазе дополнительные члены, как правило, не вносят вклад в итоговую потерю, а модель используется прежде всего для получения $\hat{Y}$, то есть предсказанной матрицы управляющих отношений.

5. Исследование форматов данных и реконструкция пайплайна

Одной из самых трудоемких задач стало выяснение, как проект формирует свои train/dev/test артефакты. На уровне интерфейса модели казалось, что данные просто загружаются из pickle, однако структура этих pickle-файлов была неочевидной.

5.1. Анализ train-данных

Файлы из каталога `labeled_data` показали, что обучение опирается на заранее подготовленные экземпляры, а не на сырые `.story`. Исторический файл `process_bert_labeled.py` позволил восстановить общую логику преобразования:

1. получение токенизированных предложений документа и абстракта;
2. преобразование слов в индексы по словарю;
3. паддинг предложений до фиксированной длины;
4. ограничение максимального числа предложений;
5. построение масок;
6. сохранение labels и исходных предложений.

5.2. Анализ dev/test-данных

Каталог `testing_final_20201012` содержал отдельную ветку пайплайна для dev и test. Были изучены:

- `handle_testing_data.py`;
- `process_for_seq2graph.py`;
- `test_seq2graph.py`;
- связанные каталоги `processed`, `processed_for_seq2graph`, `a_labeling_*`, `a_generated_*`.

Выяснилось, что dev/test-наборы проходят две стадии:

1. из каталога `.story` строится `dev_full.p` или `test_full.p`, содержащий исходные предложения документа и highlights;
2. затем этот файл переводится в `dev_for_mymodel_info.p` или `test_for_mymodel_info.p`, где предложения уже представлены индексами и масками.

6. Построение кореферентных карт без AllenNLP

Исходно проект использовал AllenNLP для построения кореферентных карт, которые затем служили структурной основой для GCN. Однако библиотека оказалась устаревшей и перестала быть надежным фундаментом для современной воспроизводимой работы.

Содержательная идея кореферентной карты в проекте такова:

1. найти все упоминания сущности;
2. объединить эти упоминания в кластер;
3. сопоставить кластер с предложениями, где упоминания встречаются;
4. взять первое упоминание как корень;
5. провести directed-ребра от предложения с первым упоминанием к остальным предложениям, где сущность повторяется.

Новая реализация `create_coreference_maps.py` сохраняет эту логику, но избавляет проект от AllenNLP. В ней были реализованы:

- нормализация упоминаний;
- приведение фраз к канонической форме;
- кластеризация повторяющихся упоминаний;
- построение directed-графов на уровне предложений;
- запись карт в формат, пригодный для последующего использования GCN.

Важным практическим результатом стало то, что кореферентные карты теперь можно воспроизводимо строить не только для train, но и для пользовательских dev/test-наборов.

6.1. Формальное описание кореферентной карты

Пусть документ состоит из последовательности предложений

$$S = \{s_1, s_2, \ldots, s_N\}.$$

Пусть также после процедуры выделения упоминаний построено множество кореферентных кластеров

$$\mathcal{C} = \{C_1, C_2, \ldots, C_K\},$$

где каждый кластер C_k содержит набор упоминаний одной и той же сущности:

$$C_k = \{m_1^{(k)}, m_2^{(k)}, \ldots, m_{r_k}^{(k)}\}.$$

Для каждого кластера определяется множество индексов предложений, в которых встречаются его упоминания:

$$I_k = \{i \mid s_i C_k\}.$$

Если множество I_k непусто, в качестве корневого предложения кластера выбирается первое по порядку вхождение:

$$r_k = \min I_k.$$

Тогда directed-кореферентные связи для данного кластера задаются множеством

$$E_k = \{(r_k, j) \mid j \in I_k,\ j \neq r_k\}.$$

Итоговый кореферентный граф документа можно записать как

$$G_{coref} = (V, E),$$

где

$$V = \{1, 2, \ldots, N\}, \qquad E = \bigcup_{k=1}^{K} E_k.$$

Если используется матричное представление графа, то ему соответствует бинарная матрица

$$A^{coref} \in \{0, 1\}^{N \times N},$$

в которой

$$A_{ij}^{coref} = \begin{cases} 1, & \exists C_k :\ i = r_k,\ j \in I_k,\ j \neq i, \\ 0, & otherwise. \end{cases}$$

Такая формализация соответствует исходной идее проекта: ребро от предложения i к предложению j трактуется как структурная подсказка о том, что предложение i инициирует или удерживает сущностную тему, продолжающуюся в j.

7. Автоматическая разметка графов на основе DistilBERT

После восстановления форматов данных потребовалось воспроизвести supervision-разметку, которая в проектной логике частично связывалась с DistilBERT.

На инженерном уровне был реализован устойчивый и воспроизводимый вариант:

1. каждое предложение кодируется `distilbert-base-uncased`;
2. из скрытых состояний вычисляется pooled-вектор предложения;
3. между предложениями строится матрица попарного косинусного сходства;
4. значение переводится из диапазона $[-1, 1]$ в диапазон $[0, 1]$;

5. диагональ принудительно устанавливается равной 1.

Этот подход не эквивалентен полноценному дообученному annotator-модулю, однако дает стабильный supervision-сигнал, пригодный для обучения и экспериментов.

Разметка на основе DistilBERT была интегрирована в разные части проекта:

- генерация labeled pickle-файлов из `.story`;
- обновление поля labels в существующем `data_info.p`;
- построение dev-входов через `process_for_seq2graph.py`;
- dev-пайплайн через `run_dev_from_story.py`.

7.1. Формальное описание DistilBERT-разметки

Пусть документ содержит N предложений

$$S = \{s_1, s_2, \ldots, s_N\}.$$

После пропуска каждого предложения через DistilBERT определяется его векторное представление

$$h_i \in R^d, \qquad i = 1, \ldots, N.$$

В текущей реализации используется mean pooling по токенам последнего скрытого слоя, маскированного attention-mask, то есть:

$$h_i = \frac{\sum_{t=1}^{T_i} m_{it} \cdot z_{it}}{\sum_{t=1}^{T_i} m_{it}},$$

где z_{it} — скрытое состояние t-го токена предложения s_i, а m_{it} — компонента маски, равная 1 для реальных токенов и 0 для паддинга.

Далее для всех пар предложений вычисляется косинусное сходство:

$$\cos(h_i, h_j) = \frac{h_i^\top h_j}{\|h_i\| \, \|h_j\|}.$$

Для получения supervision-матрицы это значение переводится в диапазон $[0, 1]$:

$$L_{ij}^{bert} = \frac{\cos(h_i, h_j) + 1}{2}.$$

После этого диагональ принудительно фиксируется:

$$L_{ii}^{bert} = 1.$$

Итоговая матрица

$$L^{bert} \in [0,1]^{N \times N}$$

используется как мягкий supervision-сигнал, где большие значения соответствуют большей семантической близости предложений.

8. Пайплайн подготовки dev и test из сырых `.story`-документов

Значительная часть работы была связана с тем, чтобы проект можно было применять к новым текстам. Для этого был выстроен воспроизводимый путь от каталога `.story` до готового модельного входа.

8.1. Сборка полного dev/test pickle

Файл `handle_testing_data.py` был расширен таким образом, чтобы:

- принимать входную директорию и выходной путь через аргументы;
- корректно извлекать content и highlights из `.story`;
- выполнять разбиение текста на предложения;
- формировать `dev_full.p` или `test_full.p`.

8.2. Преобразование в seq2graph-вход

Далее `process_for_seq2graph.py` был модифицирован так, чтобы:

- принимать произвольный pickle как источник;
- писать результат в задаваемую директорию;
- строить labels в различных режимах;
- работать как для dev, так и для пользовательских сценариев.

Итогом этой стадии являются файлы:

- `dev_for_mymodel_info.p`;
- `max_info.p`.

8.3. Оркестрация полного контура

Для объединения всех стадий был создан `run_dev_from_story.py`. Его логика позволяет независимо управлять тем, строится ли заново:

- полный pickle документа;
- seq2graph-вход;
- кореферентная карта;
- generated-директория.

Этот скрипт фактически стал внешней оболочкой всего пользовательского dev-пайплайна.

8.4. Формальное описание преобразования документа в seq2graph-вход

Пусть документ представлен как упорядоченный список предложений

$$S = \{s_1, s_2, \ldots, s_N\}.$$

После токенизации каждое предложение s_i преобразуется в последовательность токенов

$$w_i = (w_{i1}, w_{i2}, \ldots, w_{iT_i}).$$

Далее каждый токен отображается в индекс словаря:

$$x_{it} = \text{vocab}(w_{it}).$$

Для фиксированного максимального числа токенов в предложении L_{max} строится padded-представление

$$\tilde{x}_i \in N^{L_{\text{max}}},$$

а также токеновая маска

$$m_i \in \{0, 1\}^{L_{\text{max}}},$$

где единицы соответствуют реальным токенам, а нули — дописанным позициям паддинга.

Для фиксированного максимального числа предложений в документе N_{max} формируется массив документа

$$X \in N^{N_{\text{max}} \times L_{\text{max}}},$$

состоящий из padded-представлений предложений, и последовательностная маска

$$q \in \{0, 1\}^{N_{\text{max}}},$$

отмечающая, какие позиции соответствуют реальным предложениям, а какие — дописаны для выравнивания длины документа.

Именно тройка

$$(X, M, q),$$

где M — набор токеновых масок, образует основу входа для seq2graph-модели. К ней добавляются labels, исходные предложения, абстракт и другие служебные поля.

9. Поддержка пользовательских dev-наборов и управляемого инференса

Исходный `test_seq2graph.py` не был достаточно гибким для dev-сценариев. Поэтому был создан отдельный файл `dev_seq2graph.py`, который:

- принимает явный входной pickle;
- принимает полный dev pickle;
- получает путь к max-информации;
- получает путь к кореферентным картам;
- позволяет задавать модельный чекпоинт;
- сохраняет generated mind-map файлы в отдельную директорию;
- затем запускает оценку по benchmark-каталогу.

В результате был реализован полноценный сценарий controlled inference, где исследователь может варьировать:

- тексты;
- labels;
- кореферентные карты;
- benchmark-файлы;
- модельный чекпоинт.

10. Модификация `train.py` для работы с альтернативными dev-данными

Следующим этапом стало расширение `train.py`. Исходно обучение использовало стандартные dev-артефакты. Это затрудняло исследование на новых наборах.

В ходе работы были добавлены аргументы, позволяющие явно задавать:

- альтернативный `dev_input`;
- альтернативный `dev_full`;
- альтернативный `dev_max_info`;
- отдельные `coref_dev_maps`.

Такая модификация позволила сохранить общую логику обучения модели, но сделать dev-этап подменяемым. Благодаря этому проект стал пригоден для сравнения качества модели на различных типах dev-наборов без изменения основной кодовой базы обучения.

11. Система оценки качества и переработка метрик

В проекте используется нестандартный контур оценки, ориентированный не на локальную классификационную точность отдельных ребер, а на сопоставление generated и gold mind-map структур.

На практике были проанализированы три группы метрик:

- sentence-level similarity;
- keyword-level similarity;
- TTED.

11.1. Sentence-level и keyword-level метрики

В обеих метриках используется общая функция greedy-сопоставления пар, но над разными типами объектов:

- для sentence-level сравниваются строки предложений;
- для keyword-level сравниваются строки ключевых фраз.

В исходной реализации было обнаружено, что функция сравнения мутирует переданный список, удаляя уже подобранные пары. Это делало результат зависимым от порядка данных. Функция была переписана в немутирующий вариант с учетом множества уже использованных соответствий.

11.2. Диагностика проблем оценки

Были найдены и устранены следующие проблемы:

- несоответствие benchmark-каталога и generated-каталога;
- использование старых generated-файлов при переоценке;
- несогласованность путей между скриптами верхнего уровня и оценочными модулями;
- пропуски документов при частичных наборах файлов.

12. Интеграция TTED в общий оценочный контур

TTED был встроен в проект как дополнительная структурная метрика. В ходе работы:

- был установлен приоритет локального TTED-модуля из репозитория CMGN;
- были исправлены пути импорта;
- была доработана логика возврата TTED из оценочной функции;
- было добавлено усредненное значение TTED по набору;
- были обработаны случаи, когда TTED не вычисляется для конкретного файла.

Тем самым TTED стал частью регулярного контура оценки, а не отдельным экспериментальным модулем.

12.1. Содержательный смысл TTED

TTED (*Text Tree Edit Distance*) используется в проекте как метрика структурного сходства между двумя деревьями смысловой организации документа. В отличие от sentence-level и keyword-level метрик, которые сопоставляют локальные пары вершин и ребер, TTED оценивает объект более высокого уровня — дерево целиком. Это особенно важно для задачи mind-map генерации, поскольку в ряде случаев два решения могут иметь близкие локальные связи, но существенно отличаться по общей иерархической организации.

Идея метрики состоит в следующем: каждому документу сопоставляется дерево, вершины которого содержат текстовые фрагменты, а ребра отражают отношение родитель-потомок. Далее измеряется, насколько сложно преобразовать одно дерево в другое с учетом как структуры, так и смыслового расстояния между текстами узлов. Чем меньше такое расстояние, тем ближе сгенерированная карта к эталону.

12.2. Как TTED реализован в текущем проекте

В текущей реализации проекта TTED вычисляется в модуле `evaluation/evaluations.py`. В логике расчета участвуют два дерева:

- **эталонное дерево**, которое восстанавливается из benchmark-файла `.story`;
- **сгенерированное дерево**, которое строится из пар связей, полученных после работы модели и процедуры `my_generate_mindmap`.

Эталонное дерево извлекается из разметки вида `<T1>`, `<T1.1>`, `<T1.1.1>` и т.д. Эта иерархическая маркировка позволяет восстановить отношение родительского и дочернего узла напрямую по вложенности тегов. После этого дерево нормализуется: выявляются корни, достраиваются связи для сиротских корней, производится переупорядочение узлов и формируется объект `TextTree`.

Сгенерированное дерево строится иным путем. Модель предсказывает матрицу межпредложенческих связей, затем из нее собирается набор directed-пар, описывающих mindmap структуру. Эти пары преобразуются в дерево: каждому уникальному предложению сопоставляется вершина, после чего по directed-парам восстанавливаются дочерние связи. Полученная структура также нормализуется и переводится в формат `TextTree`.

12.3. Семантическая компонента TTED

Важной особенностью текущей реализации является то, что расстояние между узлами сравниваемых деревьев считается не как строго лексическая метрика, а как семантическая дистанция между эмбеддингами текстов. Для этого используется модель `sentence-transformers/` Текст каждого узла кодируется в векторное представление, после чего расстояние между двумя узлами вычисляется как косинусная дистанция:

$$d(a, b) = 1 - \cos(a, b)$$

где a и b — эмбеддинги текстов двух сравниваемых узлов.

Если нейросетевая модель эмбеддингов недоступна, в коде предусмотрен более простой резервный механизм: хэш-эмбеддинг, основанный на токенах строки. Однако основным вариантом в проекте остается sentence-transformer.

12.4. Формальное описание TTED в контексте проекта

Пусть $T_g = (V_g, E_g)$ обозначает эталонное дерево документа, восстановленное из benchmark-разметки, а $T_p = (V_p, E_p)$ — дерево, построенное по предсказанию модели. Каждая вершина $v \in V$ ассоциирована с текстовым содержимым $s(v)$, представляющим собой предложение или текстовую строку узла дерева.

Далее вводится отображение

$$\phi : s(v) \to R^d,$$

которое переводит текст узла в векторное пространство эмбеддингов. В текущей реализации проекта функция ϕ задается через sentence-transformer модель `paraphrase-distilroberta-base-v2`. Если нейросетевой энкодер недоступен, используется резервная приближенная функция хэш-эмбеддинга.

Стоимость замены одного узла другим определяется через косинусную дистанцию между эмбеддингами:

$$c_{sub}(u, v) = 1 - \cos(\phi(s(u)), \phi(s(v))).$$

Соответственно:

- чем ближе тексты узлов по смыслу, тем меньше $c_{sub}(u, v)$;
- чем сильнее различаются тексты узлов, тем ближе стоимость замены к 1.

На уровне деревьев TTED можно рассматривать как функцию

$$\mathrm{TTED}(T_p, T_g),$$

которая измеряет минимальную цену преобразования структуры T_p в структуру T_g с учетом операций над вершинами и ребрами. В библиотечной реализации, используемой в проекте, эта величина рассчитывается не вручную в явном виде, а через готовую функцию `avg_tted`, однако ее содержательная интерпретация остается той же: чем меньше значение, тем ближе предсказанная и эталонная карты как по структуре, так и по семантическому наполнению узлов.

Если обозначить множество документов тестового или dev-набора как

$$\mathcal{D} = \{D_1, D_2, \ldots, D_N\},$$

а через $\mathcal{D}^* \subseteq \mathcal{D}$ — подмножество документов, для которых TTED удалось корректно вычислить, то усредненная TTED-метрика в проекте имеет вид

$$\overline{\mathrm{TTED}} = \frac{1}{|\mathcal{D}^*|} \sum_{D_i \in \mathcal{D}^*} \mathrm{TTED}(T_p^{(i)}, T_g^{(i)}).$$

Именно эта величина выводится как усредненный структурный показатель качества по набору.

12.5. Что именно штрафует TTED

Формально TTED в текущем проекте чувствителен к трем типам расхождений:

1. **структурное расхождение:** различие в расположении узлов и родительско- дочерних отношениях между ними;
2. **семантическое расхождение:** различие между текстами сопоставляемых узлов, измеряемое через косинусную дистанцию их эмбеддингов;
3. **иерархическое расхождение:** несовпадение уровня и роли узла внутри дерева, даже если его текст частично совпадает с эталонным.

Из этого следует, что TTED является более строгой метрикой, чем обычное сравнение наборов пар связей. Два решения могут иметь похожие локальные ребра, но если они организованы в разные деревья, TTED это зафиксирует.

12.6. Режим вычисления TTED

Для сравнения двух деревьев используется функция `avg_tted`. В текущей конфигурации она вызывается с параметрами:

- `unordered=True`;
- `use_context=False`.

Это означает, что при вычислении метрики не предполагается строгий учет порядка дочерних узлов, а контекст соседних поддеревьев не вводится как дополнительный источник штрафа. Такой выбор представляется разумным именно для mind-map задачи, где важнее сама иерархическая принадлежность и близость узлов, чем жесткий порядок перечисления потомков.

12.7. Интерпретация результата

Значение TTED интерпретируется как мера структурно-семантического расхождения между сгенерированным деревом и эталонным. Чем меньше значение, тем ближе дерево модели к gold структуре. При этом метрика учитывает одновременно:

- различие в топологии дерева;
- различие в текстовом содержимом узлов;
- различие в расположении текстовых фрагментов в иерархии.

Это делает TTED качественно более строгой метрикой, чем sentence-level similarity. Даже если отдельные пары предложений совпадают локально, модель может получить худшее значение TTED, если общая организация дерева отличается от эталона.

12.8. Усредненный TTED по набору

Одной из доработок оценочного контура стало добавление метрики TTED. Изначально код мог печатать значения только для rouge-метрки. Учитывая, что rouge метрики довольно плохо улавливают структурные различия, было важно обеспечить метрику, которая бы фиксировала эти различия. TTED оказался подходящим кандидатом. После добавления логики оценка стала возвращать и отображать:

- среднюю sentence-level метрику;
- среднюю keyword-level метрику;
- средний TTED по всем документам, для которых он был успешно вычислен.

Это позволило использовать TTED не только как точечную диагностическую меру, но и как полноценный агрегированный показатель качества структурной генерации.

13. Внедрение RST-разметки как альтернативы DistilBERT

Одним из наиболее содержательных направлений работы стало внедрение риторической разметки. Цель заключалась в том, чтобы использовать не только семантическое сходство предложений, но и дискурсивную структуру текста.

13.1. Выбор парсера

Для английских текстов был выбран `IsaNLP RST Parser`. Выбор был обусловлен тем, что данный парсер:

- работает с английским языком;
- возвращает структурированное RST-дерево;
- поддерживает современную модель через внешние веса;
- сравнительно удобен для интеграции.

13.2. Реализация базового модуля RST-labels

В файле `data_pipeline/rst_labels.py` реализован полный цикл:

1. инициализация парсера;
2. вызов парсинга текста;
3. извлечение дерева и листовых EDU;

4. обход дерева с анализом ядерности;
5. формирование directed-связей между EDU;
6. сопоставление EDU предложениям;
7. построение бинарной матрицы связей между предложениями.

13.3. Логика directed-связей

В реализованной схеме:

- при `NS` единица ставится от nucleus к satellite;
- при `SN` единица ставится от nucleus к satellite;
- при `NN` directed-связь не создается, так как отношение интерпретируется как равноправное;
- если оба EDU относятся к одному предложению, связь не переносится в матрицу.

Это соответствует содержательной интерпретации labels как матрицы “кто управляет кем”, а не как матрицы любой дискурсивной связанности.

13.4. Формальное описание RST-labels на уровне предложений

Пусть RST-парсер возвращает дерево EDU-единиц

$$T^{rst} = (U, E^{rst}),$$

где $U = \{u_1, u_2, \ldots, u_M\}$ — множество элементарных дискурсивных единиц, а E^{rst} — множество отношений между ними.

Для каждого внутреннего узла дерева известен тип ядерности $\nu \in \{NS, SN, NN\}$. В текущей реализации directed-связи строятся только для случаев NS и SN:

$$u_p \to u_c \quad , u_p{-}nucleus, u_c{-}satellite.$$

При $\nu = NN$ directed-связь не создается.

Далее вводится отображение

$$\pi : U \to \{1, 2, \ldots, N\},$$

которое сопоставляет каждому EDU индекс предложения, к которому он относится. Это отображение восстанавливается либо по позиции EDU в полном тексте, либо по fallback-логике токенового перекрытия.

После этого строится sentence-level матрица

$$L^{rst} \in \{0,1\}^{N \times N},$$

где

$$L^{rst}_{ij} = \begin{cases} 1, & \exists u_p, u_c \in U : \ \pi(u_p) = i, \ \pi(u_c) = j, \ (u_p, u_c) \in E^{rst}, \ i \neq j, \\ 0, & otherwise. \end{cases}$$

На диагонали матрицы фиксируется

$$L^{rst}_{ii} = 1.$$

Таким образом, итоговая матрица отражает directed-управление между предложениями, выведенное из риторической структуры текста.

14. Принципы агрегации EDU-структуры на уровень предложений

Ключевая методическая проблема RST-разметки связана с несовпадением уровней анализа: парсер работает на уровне EDU, а модель CMGN — на уровне предложений.

Для согласования этих уровней была реализована следующая процедура:

1. предложения документа склеиваются в единый текст;
2. для каждого EDU ищется позиция его текста в общем документе;
3. по позиции определяется предложение, которому EDU соответствует;
4. если позиционное соответствие не найдено, используется fallback по токеновому пересечению;
5. directed-связь между EDU переносится на directed-связь между предложениями.

Достоинство такого подхода состоит в его совместимости с уже существующей модельной архитектурой. Ограничение заключается в том, что часть внутрипредложенческой дискурсивной информации неизбежно теряется.

14.1. Формальная схема отображения EDU в предложения

Пусть документ состоит из предложений $S = \{s_1, \ldots, s_N\}$, а объединенный текст документа обозначим как

$$\Sigma = s_1 \oplus s_2 \oplus \ldots \oplus s_N,$$

где $\oplus$ обозначает конкатенацию через пробел.

Для каждого предложения s_i задается интервал позиций

$$I_i = [\ell_i, r_i)$$

внутри строки Σ. Если для EDU u_k с текстом t_k найдена позиция начала $p_k \in \Sigma$, то предложение определяется как

$$\pi(u_k) = i \quad \Leftrightarrow \quad p_k \in I_i.$$

Если точного положения найти не удается, используется приближенная функция сходства

$$\text{overlap}(u_k, s_i) = |\text{Tok}(u_k) \cap \text{Tok}(s_i)|\,,$$

и выбирается предложение

$$\pi(u_k) = \arg\max_{i \in \{1,\ldots,N\}} \text{overlap}(u_k, s_i).$$

Именно это отображение π делает возможным перенос RST-отношений с уровня EDU на уровень предложений без изменения архитектуры модели.

15. Анализ проблем, выявленных в ходе работы, и способы их устранения

В процессе модернизации были систематически выявлены и устранены следующие классы проблем.

15.1. Проблемы зависимостей

- устаревание AllenNLP и необходимость его исключения;
- отсутствие некоторых NLTK-ресурсов;
- рассогласование между различными реализациями модулей оценки.

15.2. Проблемы данных

- невозможность восстановить оригинальные pickle-структуры из-за отсутствия документации;
- невозможность обучения на новых данных в связи с отсутствием зафайьюненго DistilBERT-annotator-модуля;
- отсутствие или недоступность словаря токенов;
- несовпадение разметки и оригинального текста (в оригинальном тексте отсутствовал заголовок).

15.3. Проблемы оценки

- чтение старых generated-карт;
- использование неправильного benchmark-каталога;
- нестабильность sentence-level метрики из-за мутации списка;
- использовались старые методы оценки, не учитывающие структуру карт.

Решение этих проблем фактически составило значительную часть практической работы, потому что именно они мешали проекту быть воспроизводимым.

16. Результаты проделанной работы

По итогам проведенной деятельности можно выделить несколько групп результатов.

16.1. Инженерные результаты

- реализован воспроизводимый путь от `.story` до модельного входа;
- построен новый механизм кореферентных карт;
- внедрены новые режимы supervision-разметки;
- разработан отдельный dev-пайплайн;
- обучение и инференс стали принимать альтернативные наборы данных;
- система оценки была стабилизирована и расширена.

16.2. Исследовательские результаты

- проект подготовлен к сравнению различных способов автоматической разметки;
- создана основа для анализа влияния кореферентных и риторических структур на качество графов;
- появилась возможность ставить новые эксперименты на пользовательских текстах.

16.3. Экспериментальные измерения

Ниже приведены сводные количественные результаты, полученные в ходе работы с проектом и включенные в текст статьи как экспериментальное обоснование предложенных модификаций. Данный блок фиксирует не только финальные значения метрик, но и позволяет проследить, как отдельные инженерные решения влияли на качество модели. Для интерпретации значений важно учитывать направление оптимизации:

- для `sent` и `word` большее значение соответствует лучшему качеству;
- для `tted` меньшее значение соответствует более близкой структурной карте.

16.3.1. Метрики по статье

Таблица 1: Значения метрик, зафиксированные в материале статьи

Конфигурация	sent	word
Full Model	0.443	0.437
w/o rl	—	—
w/o gcn	0.423	0.421
w/o gcl	0.217	0.246
w/o gcl + w/o gcn	0.416	0.416

Для конфигурации `w/o rl` численные значения в статье не были предоставлены, поэтому в таблице они оставлены как отсутствующие. Замеры проведены на тестовом наборе данных. Из схемы видно, что `Full Model` в статье служит основной точкой сравнения, а наиболее сильная деградация происходит при удалении `gcl`, где значения `sent` и `word` падают до 0.217 и 0.246 соответственно. Конфигурация `w/o gcn` также уступает полной модели, но остается заметно лучше, чем `w/o gcl`. При этом совмещенная абляция `w/o gcl + w/o gcn` показывает значения 0.416 и 0.416, то есть оказывается лучше, чем каждая из этих двух одиночных абляций по отдельности, но все равно не достигает уровня `Full Model`. Это означает, что в статье влияние компонентов не является линейным: удаление одного блока может ухудшать модель сильнее, чем одновременное удаление нескольких блоков, поэтому интерпретировать такие абляции необходимо как взаимодействующие, а не как независимые факторы.

16.3.2. Метрики по коду на test-наборе

Таблица 2: Результаты абляций по метрикам кода на test-наборе

Конфигурация	sent	word	tted
Full Model	0.311	0.405	0.483
w/o rl	0.188	0.262	0.679
w/o gcn	0.223	0.275	0.570
w/o gcl	0.199	0.262	0.672
w/o gcl + w/o gcn	0.168	0.190	0.686

На test-наборе полная модель показывает лучший результат сразу по всем трем метрикам в рассматриваемой группе абляций. Удаление `rl`, `gcn` и `gcl` во всех случаях ухудшает качество, причем наиболее сильное падение наблюдается для совмещенной абляции `w/o gcl + w/o gcn`. Это позволяет сделать важный вывод: на test-наборе полная конфигурация архитектуры действительно оправдана, а локальные упрощения модели не приводят к улучшению ни по текстовым, ни по структурным метрикам.

16.3.3. Метрики по коду на dev-наборе

Таблица 3: Результаты абляций по метрикам кода на dev-наборе

Конфигурация	sent	word	tted
Full Model	0.301	0.387	0.525
w/o rl	0.309	0.398	0.530
w/o gcn	0.214	0.270	0.607
w/o gcl	0.312	0.411	0.519
w/o gcl + w/o gcn	0.291	0.363	0.532

На dev-наборе картина менее однозначна. Конфигурация `w/o gcl` показывает лучшее значение по всем трем метрикам относительно полной модели: `sent`=0.312, `word`=0.411 и `tted`=0.519. Вариант `w/o rl` также улучшает текстовые метрики по сравнению с baseline, но проигрывает по `tted`. При этом удаление `gcn` приводит к резкой деградации как на dev, так и на test, что подтверждает ключевую роль графовой компоненты. Таким образом, dev-эксперименты показывают, что отдельные упрощения могут локально улучшать качество на валидации, но эти улучшения не переносятся на test-набор.

16.3.4. Замеры на разных вариантах dev-текстов

Таблица 4: Влияние варианта dev-текста на итоговые метрики

Конфигурация	sent	word	tted
Baseline	0.301	0.387	0.525
Baseline with my preprocessing	0.309	0.388	0.536
with title	0.311	0.406	0.498

Из результатов видно, что вариант `with title` дал наиболее сильный word-level результат и одновременно наименьшее значение TTED. Это означает, что добавление заголовка в анализируемый текст в данном эксперименте улучшило как лексико-семантическое

соответствие, так и общую структурную близость дерева. С практической точки зрения это важно, поскольку показывает чувствительность системы к составу входного текста: даже относительно небольшое изменение информационного контекста документа может привести к перестройке предсказываемой карты и, следовательно, к изменению финального качества.

16.3.5. Собственные карты кореферентности

Таблица 5: Влияние собственных карт кореферентности на качество

Конфигурация	sent	word	tted
Baseline	0.303	0.392	0.528
coreference maps on train only	0.314	0.403	0.525
coreference maps on validation only	0.309	0.410	0.522
coreference maps on train and validation	0.289	0.384	0.545

Эти измерения показывают, что использование собственных кореферентных карт только на train или только на validation может улучшать результат относительно baseline, тогда как одновременная замена карт на train и validation в данном эксперименте привела к ухудшению как sentence/word метрик, так и TTED. Иными словами, качество кореферентного prior зависит не только от самого алгоритма его построения, но и от того, на какой части набора он используется. Это согласуется с общим наблюдением, что графовый контекст может быть полезен лишь при условии согласованности с распределением данных, на котором обучается и валидируется модель.

16.3.6. Собственный кодировщик для labels на обучении

Таблица 6: Сравнение supervision-сигналов при обучении

Конфигурация	sent	word	tted
Baseline	0.303	0.392	0.528
DistilBert	0.248	0.299	0.732
RST Networks	0.314	0.384	0.584
RST Networks only NS	0.317	0.390	0.560

Сравнение показывает, что в данной конфигурации DistilBERT-supervision дал худший результат по всем трем метрикам. RST-разметка оказалась заметно сильнее, причем вариант `RST Networks only NS`, в котором используются только направленные отношения nucleus $\rightarrow$ satellite, показал лучший результат среди RST-схем и оказался близок

к baseline. Это наблюдение подтверждает исходную гипотезу о том, что для задачи построения направленной mind map более полезным оказывается именно асимметричный supervision-сигнал, в котором явным образом кодируется отношение управления между узлами, тогда как равноправные риторические связи менее полезны для построения ориентированного графа.

Таблица 7: Сравнение RST-кодировщиков по метрикам кода на dev-наборе

Конфигурация	sent	word	tted
RST Networks	0.310	0.381	0.591
RST Networks only NS	0.318	0.386	0.569

Таблица 8: Сравнение RST-кодировщиков по метрикам кода на test-наборе

Конфигурация	sent	word	tted
RST Networks	0.315	0.392	0.581
RST Networks only NS	0.319	0.399	0.543

Дополнительные измерения на dev и test подтверждают, что вариант `RST Networks only NS` устойчиво превосходит полный RST-вариант по всем трем метрикам. На dev-наборе он улучшает `sent` с 0.310 до 0.318, `word` с 0.381 до 0.386 и снижает `tted` с 0.591 до 0.569. На test-наборе картина сохраняется: `sent` возрастает с 0.315 до 0.319, `word` — с 0.392 до 0.399, а `tted` уменьшается с 0.581 до 0.543. Следовательно, если использовать RST как источник supervision-сигнала, то именно directed-версия с сохранением только отношений `NS/SN` является более предпочтительной.

16.3.7. Сводная интерпретация замеров

Собранные результаты позволяют сделать несколько предварительных выводов.

1. Значения, приведенные в статье, качественно согласуются с общей логикой воспроизводимых экспериментов: `Full Model` выступает базовой и наиболее сильной точкой сравнения, а удаление ключевых компонентов приводит к деградации.

2. На test-наборе полная модель является лучшей среди абляций по всем трем метрикам, что подтверждает оправданность полной архитектуры в финальной постановке.

3. На dev-наборе удаление `gcl` дает локальное улучшение по всем метрикам, однако это улучшение не переносится на test; следовательно, по одной лишь валидации делать вывод о превосходстве упрощенной конфигурации нельзя.

4. Удаление `gcn` систематически ухудшает качество и на dev, и на test, поэтому графовая компонента является одним из наиболее значимых элементов модели.

5. Среди альтернативных supervision-сигналов DistilBERT показал наихудшие результаты, тогда как RST-разметка оказалась существенно более перспективной, особенно в варианте `only NS`.

6. Внутри семейства RST-кодировщиков вариант `RST Networks only NS` стабильно превосходит полный RST-вариант на dev и test, что подтверждает полезность именно направленных риторических отношений для задачи построения ориентированной карты.

7. Эксперименты с собственными картами кореферентности показывают, что train-only вариант улучшает `sent`, validation-only вариант дает лучшие `word` и `tted`, а одновременная замена карт на train и validation ухудшает все показатели.

8. На численные результаты существенно влияет и сама текстовая форма входа: вариант `with title` показывает наилучшие значения по `word` и `tted` среди экспериментов на разных dev-текстах.

9. В совокупности это означает, что качество mind map generation определяется не одной архитектурной настройкой, а согласованной работой нескольких блоков: предобработки текста, кореферентного prior, supervision-сигнала и графовой модели.

16.4. Практическая значимость

Практическая значимость работы состоит в том, что проект больше не ограничен заранее подготовленным набором артефактов. Теперь он может использоваться как гибкая исследовательская среда для:

- проверки новых supervision-сигналов;
- генерации карт по новым текстам;
- сравнения различных схем preprocessing;
- структурной оценки результатов.

17. Список ключевых модифицированных компонентов проекта

Компонент	Назначение и роль в модернизации
`create_coreference_maps.py`	Новая реализация построения кореферентных карт без AllenNLP.
`testing_final_20201012/handle_testing_data.py`	Подготовка полного dev/test pickle из `.story`.
`testing_final_20201012/process_for_seq2graph.py`	Перевод dev/test pickle в формат seq2graph-модели с поддержкой нескольких режимов labels.
`testing_final_20201012/dev_seq2graph.py`	Выделенный сценарий dev-инференса с generated- и benchmark-поддержкой.
`testing_final_20201012/test_seq2graph.py`	Расширение стандартного test-инференса.
`train.py`	Поддержка альтернативных dev-входов и отдельного набора кореферентных карт.
`evaluation/evaluations.py`	Стабилизация sentence-level метрики и интеграция TTED.
`data_pipeline/run_dev_from_story.py`	Оркестрация полного dev-пайплайна.
`data_pipeline/annotate_distilbert.py`	Автоматическая разметка `.story` каталогов через DistilBERT.
`data_pipeline/annotate_from_data_info.py`	Подмена label-поля в data_info через DistilBERT.
`data_pipeline/rst_labels.py`	Базовый модуль RST-разметки на уровне предложений.
`data_pipeline/annotate_rst_from_data_info.py`	Внедрение RST-labels в существующие pickle-структуры.
`data_pipeline/annotate_rst.py`	Построение train/dev labeled pickle-файлов из `.story` на основе RST.
`data_pipeline/inspect_dev_p.py`	Диагностика внутренней структуры pickle-данных.

17.1. Directional Discourse Supervision for Knowledge-Map Generation

Одним из наиболее содержательных исследовательских выводов, вытекающих из проделанной работы, является несоответствие между природой целевой задачи и формой части используемого supervision-сигнала. В проекте CMGN требуется построить ориентированный граф управляющих связей между предложениями документа. Формально модель предсказывает матрицу

$$\hat{Y} \in [0,1]^{N \times N},$$

где элемент $\hat{y}_{ij}$ характеризует степень того, насколько предложение s_i структурно управляет предложением s_j. Следовательно, сама постановка задачи является направленной: отношение между s_i и s_j в общем случае несимметрично.

При этом один из использованных в проекте источников автоматической разметки — матрица попарного косинусного сходства предложений, построенная на основе DistilBERT-представлений. Такой сигнал по своей природе отражает преимущественно семантическую близость, а не иерархическую или риторическую направленность. В результате supervision вида

$$L_{ij}^{\text{bert}} = \frac{\cos(h_i, h_j) + 1}{2}$$

задает мягкое симметричное сходство между предложениями, но не кодирует различие между отношениями типа “источник $\rightarrow$ уточнение” и “уточнение $\rightarrow$ источник”. Для задачи построения ориентированной карты знаний это является принципиальным ограничением.

Экспериментальные наблюдения подтверждают данную гипотезу. В отчете было показано, что DistilBERT-разметка дает худшие результаты по сравнению с направленными источниками supervision, тогда как риторическая разметка, особенно в ограниченном режиме `RST only NS`, демонстрирует более устойчивое качество. Это указывает на то, что для построения иерархической карты знаний важен не столько сам факт семантической близости предложений, сколько наличие асимметричного структурного сигнала, отражающего отношение ядра и зависимого фрагмента.

На этой основе можно сформулировать исследовательскую гипотезу о *направленном дискурсивном supervision*. Пусть для документа строятся несколько вспомогательных графов:

$$A^{\text{rst}} \in \{0,1\}^{N\times N}, \qquad A^{\text{coref}} \in \{0,1\}^{N\times N}, \qquad A^{\text{sem}} \in [0,1]^{N\times N},$$

где A^{rst} задает directed-связи, полученные из риторической структуры текста, A^{coref} — directed-кореферентные переходы между предложениями, а A^{sem} — мягкий семантический prior.

Тогда совокупный teacher-graph может быть задан в виде

$$A^{\text{teach}} = \alpha A^{\text{rst}} + \beta A^{\text{coref}} + \gamma A^{\text{sem}},$$

где коэффициенты удовлетворяют условию

$$\alpha > \beta > \gamma \geq 0,$$

что отражает более высокую значимость направленного дискурсивного сигнала по сравнению с симметричной семантической близостью.

Вместо покомпонентной регрессии только к матрице labels можно использовать ком-

бинированную целевую функцию

$$L = \lambda_1 L_{\text{dir}} + \lambda_2 L_{\text{rank}} + \lambda_3 L_{\text{cons}},$$

где L_{dir} — directed-loss для восстановления ориентированных ребер, L_{rank} — ranking-loss, обеспечивающая более высокий score для правильного родительского предложения по сравнению с альтернативами, а L_{cons} — регуляризатор согласованности между несколькими структурными представлениями.

Например, directed-компонента может быть задана как взвешенная бинарная кросс-энтропия:

$$L_{\text{dir}} = -\sum_{i \neq j} w_{ij} \left(A_{ij}^{\text{teach}} \log \hat{y}_{ij} + (1 - A_{ij}^{\text{teach}}) \log(1 - \hat{y}_{ij}) \right),$$

где веса w_{ij} могут усиливать вклад риторически наиболее значимых переходов.

Ранжирующая компонента может быть записана в виде

$$L_{\text{rank}} = \sum_{j=1}^{N} \sum_{n \neq p_j^*} \max(0,\ m - \hat{y}_{p_j^* j} + \hat{y}_{nj}),$$

где p_j^* — правильный родитель узла j, а $m > 0$ — margin. Такая формулировка особенно уместна в задаче построения карты знаний, поскольку модель должна не просто повышать значения на “хороших” ребрах, а выбирать структурно предпочтительного родителя для каждого предложения.

Наконец, компоненту согласованности можно ввести как штраф за расхождение между различными структурными источниками:

$$L_{\text{cons}} = \sum_{i,j} \left| \hat{y}_{ij}^{\text{rst}} - \hat{y}_{ij}^{\text{coref}} \right|.$$

Научная новизна такого подхода состоит в переходе от симметричного similarity-based supervision к *асимметричному структурному supervision*, согласованному с природой самой задачи. В этой постановке модель обучается восстанавливать не просто семантическую близость предложений, а directed-отношения типа “ядро → развитие”, “первое упоминание → повторное упоминание”, “структурный центр → зависимый фрагмент”. Тем самым supervision становится ближе к содержательной цели генерации карты знаний.

17.2. TTED-Aware Training for Hierarchical Knowledge Maps

Второе перспективное направление исследований связано с несоответствием между функцией потерь, используемой при обучении модели, и метрикой, по которой затем оценивается итоговое качество построенной карты знаний. В текущей реализации базовая supervised-компонента задается как среднеквадратичное отклонение между предсказан-

ной и целевой матрицами связей:

$$L_{\text{sup}} = \frac{1}{N^2} \sum_{i=1}^{N} \sum_{j=1}^{N} (\hat{y}_{ij} - y_{ij})^2.$$

Такое обучение является по существу edge-wise: модель оптимизирует локальное соответствие отдельных элементов матрицы.

Однако итоговая оценка в проекте существенно богаче. Помимо sentence-level и keyword-level показателей используется TTED — метрика, сравнивающая generated и gold mind-map структуры как деревья. В отчете подчеркивается, что TTED штрафует не только локальное несовпадение ребер, но и более сложные расхождения:

- структурные ошибки в родительско-дочерних отношениях;
- семантическое несоответствие текстов сопоставляемых узлов;
- иерархическое несоответствие, когда узел оказывается на неверном уровне дерева.

Следовательно, возникает принципиальный *train–evaluation mismatch*: модель обучается как предсказатель матрицы попарных связей, а оценивается как генератор иерархической смысловой структуры. Из этого естественно вытекает гипотеза о необходимости *TTED-aware training*, то есть обучения, в котором функция потерь приближена к тем типам ошибок, которые действительно фиксирует TTED.

Наиболее реалистичной формой такого подхода является введение структурного суррогата, который не требует прямой дифференциации TTED, но учитывает его содержательный смысл. Пусть для каждого узла j модель задает распределение вероятностей по возможным родителям:

$$p(i \mid j) = \frac{\exp(z_{ij})}{\sum_{k \neq j} \exp(z_{kj})},$$

где z_{ij} — score связи $i \to j$ до нормировки.

Если p_j^* обозначает правильного родителя узла j в gold-дереве, то parent-level компоненту можно определить как

$$L_{\text{parent}} = -\sum_{j=1}^{N} \log p(p_j^* \mid j).$$

В отличие от покомпонентной MSE, такая функция потерь непосредственно оптимизирует корректность выбора родителя, а значит, лучше согласуется с деревообразной природой итогового объекта.

Поскольку TTED чувствителен и к иерархическому положению узлов, разумно дополнительно учитывать глубину узла в дереве. Пусть d_j^* — глубина узла j в gold-структуре,

а $\hat{d}_j$ — предсказание модели. Тогда можно ввести depth-loss:

$$L_{\text{depth}} = \sum_{j=1}^{N} (\hat{d}_j - d_j^*)^2.$$

Альтернативно глубина может рассматриваться как дискретная метка, и тогда применяется классификационная форма соответствующей функции потерь.

Еще одна важная составляющая TTED связана с сохранением правильной предковой структуры. Даже если локальный родитель выбран почти корректно, ошибка в верхней части дерева может привести к изменению целого пути узла к корню. Для учета этого эффекта можно задать бинарную матрицу предковых отношений

$$A^* \in \{0, 1\}^{N \times N},$$

где

$$A^*_{aj} = \{\, 1 \,, aj, 0, .$$

Тогда ancestor-consistency loss принимает вид

$$L_{\text{anc}} = -\sum_{a,j} \left[A^*_{aj} \log \hat{A}_{aj} + (1 - A^*_{aj}) \log(1 - \hat{A}_{aj}) \right],$$

где $\hat{A}_{aj}$ — предсказанная моделью вероятность предкового отношения.

Итоговая TTED-aware функция потерь может быть задана как

$$L_{\text{TTED}} = \lambda_1 L_{\text{edge}} + \lambda_2 L_{\text{parent}} + \lambda_3 L_{\text{depth}} + \lambda_4 L_{\text{anc}},$$

где L_{edge} может совпадать со стандартной edge-level loss, а остальные компоненты приближают структурные аспекты TTED.

Содержательный смысл данной конструкции состоит в следующем. Обычная edge-level оптимизация отвечает на вопрос: *верно ли предсказано отдельное ребро?* Напротив, TTED-aware обучение отвечает на более сильный вопрос: *верно ли восстановлена иерархическая роль узла в дереве?* Именно это делает такой подход особенно уместным для генерации карт знаний, где качество определяется не только наличием связей, но и тем, как эти связи организованы в глобальную структуру.

Более сильная, но и более сложная версия данного направления состоит в переходе к minimum-risk training с reward, основанным непосредственно на TTED. Если $\hat{T}$ — дерево, сгенерированное моделью, а T^* — gold-дерево, то можно задать риск

$$L_{\text{risk}} = E_{\hat{T} \sim p_\theta(T|X)}[\text{TTED}(\hat{T}, T^*)].$$

На практике эта величина может аппроксимироваться по выборке candidate trees:

$$L_{\text{risk}} \approx \sum_{k=1}^{K} p_\theta(\hat{T}_k \mid X)\,\text{TTED}(\hat{T}_k, T^*).$$

Такая постановка концептуально особенно хорошо согласуется с архитектурой CMGN, поскольку в проекте уже присутствует RL-компонента в общей функции потерь. Следовательно, TTED может быть интерпретирован как более содержательный reward, ориентированный не на внешнее текстовое сходство, а на правильность итоговой структуры карты знаний.

С научной точки зрения основное достоинство TTED-aware training заключается в том, что оно устраняет разрыв между тем, чему модель обучается, и тем, по чему она затем оценивается. Если стандартная MSE учит модель приближать элементы матрицы попарных связей, то TTED-aware подход учит ее восстанавливать глобально согласованную иерархическую организацию документа. Именно это делает данное направление естественным и сильным кандидатом для последующей научной публикации.

18. Заключение

В ходе работы над проектом CMGN была проведена масштабная реконструкция и модернизация его исследовательской инфраструктуры. Работа охватила почти весь жизненный цикл системы: предварительную обработку данных, построение кореферентных карт, автоматическую разметку, dev/test-инференс, обучение, диагностику и оценку качества.

Главный результат заключается в том, что проект перестал быть закрытой системой, привязанной к заранее подготовленным артефактам. Он превратился в гораздо более прозрачную, модульную и пригодную для дальнейших исследований платформу. В проекте были реализованы:

- воспроизводимый пайплайн от `.story` до инференса;
- новая схема построения кореферентных карт;
- поддержка гибких источников labels;
- управляемый dev-инференс;
- подмена dev-наборов в обучении;
- стабилизированная оценка и поддержка TTED;
- RST-разметка как новая исследовательская ветка supervision.

Тем самым была заложена полноценная основа для дальнейших сравнительных экспериментов, где могут сопоставляться различные способы формирования структурной

информации о тексте. С практической точки зрения проект стал существенно удобнее для работы с новыми данными, а с научной точки зрения получил инструменты для исследования того, какой тип межпредложенческой структуры лучше соответствует задаче генерации mind-map представлений.